

算法设计与分析作业六

XXXXXXXXXX XXX

2026 年 5 月 25 日

题 1 (8.1).

解答. 设 G_1 和 G_2 的邻接矩阵分别为 $A = (a_{ij})$ 和 $B = (b_{ij})$ 。由于两个图的顶点集相同，并且输入已经按同一顶点顺序给出，只需检查任意不同顶点对在两个图中是否恰好有一条边。

算法如下：对所有 $1 \leq i < j \leq n$ ，比较 a_{ij} 和 b_{ij} 。若存在 $a_{ij} = b_{ij}$ ，则输出 “No”；若所有这样的顶点对都满足 $a_{ij} \neq b_{ij}$ ，则输出 “Yes”。显然算法只做

$$\frac{n(n-1)}{2}$$

次比较，故最坏情况下时间复杂度为 $\Theta(n^2)$ 。

下面说明该阶也是下界。任意判定算法若没有检查某个顶点对 (i, j) ，则对两个只在该位置不同的输入无法区分：一种输入中两个矩阵在该位置互补，另一种输入中两个矩阵在该位置相同，而算法的执行过程完全一样，却应给出不同答案。因此任一算法在最坏情况下至少要检查所有 $\frac{n(n-1)}{2}$ 个顶点对，时间复杂度下界为 $\Omega(n^2)$ 。□

题 2 (8.2).

解答. 采用 Horner 法从高次项到低次项依次计算。令

$$P_1(x) = a_n,$$

并对 $i = 2, 3, \dots, n+1$ 定义

$$P_i(x) = a_{n-i+1} + xP_{i-1}(x).$$

最后得到

$$P_{n+1}(x) = a_0 + a_1x + \dots + a_nx^n = P(x).$$

每一步只需要一次乘法和一次加法，所以总时间为 $\Theta(n)$ ，占用额外空间为 $O(1)$ 。

证明最优性。考虑任一算法，若某个系数 a_i 在计算过程中完全没有被处理，则把输入中的 a_i 改成另一个值后，算法执行过程不变，但由于题设 $x \neq 0$ ，多项式值一般会发生改变，算法不可能同时正确。因此每个系数都至少需要被访问一次，时间下界为 $\Omega(n)$ 。上述算法达到该下界，故在最坏情况下是最优的。□

题 3 (8.4).

解答. (1) 若顺序归并, 先令 $L = L_1$, 再依次执行

$$L \leftarrow L \cup L_j, \quad j = 2, 3, \dots, k,$$

其中每次用通常的二路归并。第 j 次归并时, 前面已经合并出的表长为 $(j-1)n/k$, 新表长为 n/k , 所以比较次数为 $O(jn/k)$ 。于是总时间为

$$\sum_{j=2}^k O\left(\frac{jn}{k}\right) = O(nk).$$

(2) 更好的做法是二分归并。每轮把当前的有序表两两归并, 表的个数约减半, 直到只剩一个表。每一轮所有元素总共只参加一次归并, 所以一轮时间为 $O(n)$; 轮数为 $\lceil \log k \rceil$, 故最坏情况下时间复杂度为 $O(n \log k)$ 。

(3) 证明下界。把所有输入按比较过程形成决策树。每个输入表有 n/k 个元素, 保持每个表内部顺序不变时, 可能的总排列数为

$$\frac{n!}{((n/k)!)^k}.$$

因而决策树至少有这么多片叶子, 深度至少为

$$\log \frac{n!}{((n/k)!)^k} = \Theta(n \log n - k \cdot (n/k) \log(n/k)) = \Theta(n \log k).$$

因此任何基于比较的算法最坏情况下都需要 $\Omega(n \log k)$ 次比较。二分归并达到该下界, 所以是渐近最优的。

□

题 4 (8.9).

解答. (1) 先用线性时间选择算法找出 L 中第 m 小的元素 a 。再扫描一遍 L , 把所有小于 a 的元素收集出来。由于 $m = o(n/\log n)$, 对这 $m-1$ 个元素排序需要

$$O(m \log m) = o(n)$$

时间。最后按升序输出这些元素以及 a 。总时间为

$$O(n) + o(n) = O(n).$$

(2) 若存在一个 $O(n)$ 时间算法能在 $m = \omega(n/\log n)$ 时完成该任务, 则它也能在 $O(n)$ 时间内把任意给定的 m 个数排序: 把这 m 个数与另外 $n-m$ 个都比它们大的辅助数放在同一表中, 运行该算法, 输出的前 m 小元素正好就是原来 m 个数的有序序列。可是基于比较的排序需要 $\Omega(m \log m)$ 时间, 而当 $m = \omega(n/\log n)$ 时有

$$m \log m = \omega(n),$$

与 $O(n)$ 矛盾。因此此时不存在基于比较的 $O(n)$ 时间算法。

□

题 5 (8.10).

解答. 设要求 n 个互异数中的第 k 小元素。为了确定答案，算法最终必须知道至少 $k - 1$ 个元素小于它，并且至少 $n - k$ 个元素大于它。先考虑小于答案的元素。若某个小于答案的元素从未在比较中输给任何元素，则它仍可能是第 k 小元素，答案无法确定；因此至少要通过 $k - 1$ 次比较给这些元素各分配一次“小于答案”的证据。同理，为了排除大于答案的元素，至少需要 $n - k$ 次比较给它们各分配一次“大于答案”的证据。

另外，答案元素本身必须至少参加

$$\min\{k - 1, n - k\}$$

次比较。否则，与它比较过的较小元素不足 $k - 1$ 个，或较大元素不足 $n - k$ 个，总可以构造另一组与已有比较结果一致的输入，使答案发生改变。

因此任何基于比较的算法最坏情况下至少需要

$$(k - 1) + (n - k) + \min\{k - 1, n - k\} = n + \min\{k, n - k + 1\} - 2$$

次比较。这就是所要求的下界。

□